

Тестовое задание: Frontend Developer

Проект: SPA для работы с грузовыми аукционами по OpenAPI-схеме.

Контекст

Нужно создать frontend-проект по готовой OpenAPI-схеме аукционов.

Схема API передаётся кандидату как источник правды: `openapi.auctions.v0.json`

AI использовать можно и нужно, но кандидат отвечает за архитектуру, качество кода, проверку результата и объяснение решений.

Цель

Сделать SPA для работы с грузовыми аукционами: список аукционов, детальная страница, история ставок и установка своей ставки.

Обязательный стек

- React
- TypeScript
- Vite
- TanStack Router
- TanStack Query
- React Hook Form + Zod
- MSW
- Feature-Sliced Design
- MobX или Zustand для точечного клиентского UI-state

Готовые компонентные библиотеки и визуальный подход — на усмотрение кандидата. Важно, чтобы интерфейс был аккуратным, адаптивным и удобным для проверки.

API

Использовать endpoints из OpenAPI-схемы:

- POST `/auctions/list` — список аукционов
- GET `/auctions/{auctionUuid}` — детальная информация
- GET `/auctions/{auctionUuid}/bets` — список ставок
- POST `/auctions/{auctionUuid}/bets` — установить ставку

Backend писать не нужно. Нужно реализовать MSW-моки, которые соответствуют схеме и реально меняют состояние после mutations.

Кандидат должен гарантировать точное соблюдение контрактов OpenAPI-схемы: структуры запросов/ответов, enum-значений, nullable-полей, ошибок и edge cases.

Маршруты

Реализовать страницы списка аукционов, карточку детального просмотра, карточку установки ставки. Режим установки ставки должен открываться по ссылке.

Список аукционов

На странице списка должны быть:

- загрузка через TanStack Query
- пагинация
- skeleton, empty и error states
- prefetch детальной страницы по intent/hover

- фильтры с синхронизацией в URL search params либо в localStorage
- Zod-валидация search params с безопасными fallback-значениями
- адаптация под desktop/mobile

Минимальные фильтры

- cargo_num
- status
- statuses
- auc_type
- load_city — из мок-словаря городов
- unload_city — из мок-словаря городов
- дата погрузки от/до
- is_available
- is_bidder
- цена от/до

Карточка аукциона

Карточка аукциона должна показывать:

- номер заявки
- тип аукциона: Request, Up, Down, FixPrice
- статус аукциона
- торговый статус пользователя: Leading, Losing, Winner, etc.
- маршрут погрузка → выгрузка
- даты погрузки/разгрузки
- груз: название, вес, объём, тип кузова
- текущую цену, цену за км, шаг ставки
- флаг "моя ставка есть / нет"
- primary action: "Сделать ставку", "Изменить ставку", "Смотреть ставки" или disabled-состояние

Детальная страница

Детальная страница должна использовать GET /auctions/{auctionUuid} и показывать:

- основные данные аукциона
- организатора
- контакты, если они не скрыты
- маршрут со всеми точками
- груз и требования к ТС
- условия оплаты
- параметры торгов
- текущую цену, доступную цену, min/max/step
- состояние своей ставки
- ограничения из DTO: can_set_bet, hide_bets_history, hide_points_address_and_contacts, no_view_cargo_price

Ставки

Страница или вкладка ставок должна использовать GET /auctions/{auctionUuid}/bets.

Показать:

- список ставок
- количество участников
- цену с НДС / без НДС
- перевозчика
- место в рейтинге
- признак победителя
- признак отменённой ставки
- причину отмены, если есть
- empty state, если ставок нет
- состояние, когда история ставок скрыта через hide_bets_history

Бизнес-действие: установка ставки

Реализовать форму "Сделать ставку".

Требования:

- доступность формы зависит от `trading.can_set_bet`
- React Hook Form + Zod
- цена обязательна и больше 0
- учитывать `min`, `max`, `step`, если эти поля есть в detail DTO
- показывать подсказку по доступной цене и шагу ставки
- mutation вызывает `POST /auctions/{auctionUuid}/bets`
- после успеха инвалидируются `list/detail/bets` query
- MSW-store должен обновить текущую цену, статус пользователя и список ставок
- success/error toast
- обработка 422 validation error

AI Usage

Добавить `AI_USAGE.md`:

- какие части делались с AI
- какие решения кандидат принял сам
- какие AI-предложения были отклонены
- какие места кандидат проверял особенно внимательно
- какие риски остались
- что бы кандидат улучшил при наличии ещё одного дня

Проверка

Проект должен запускаться локально.

Кандидат сам выбирает способ проверки результата и описывает его в README: что именно проверял, какие сценарии прошёл, какие ограничения остались.

Желательно добавить минимальные тесты на чистую логику, например:

- search params parsing
- request builder
- ViewModel-мапперы
- validation schema ставки

Что сдавать

- ссылка на репозиторий
- README с запуском